

Department of Statistics & Computer Science
University of Kelaniya
Academic Year -2024/2025 (Semester II)

COSC 32133/BECS 32263

Full-Stack Software Development

Lab Sheet 05

Spring Boot CRUD Application with MySQL, Lombok & GitHub

Generate a Spring Boot project with **Spring Initializer** using the **MySQL Driver** and **Lombok** dependencies. Build an application that collects user input through Postman, stores it in a **MySQL database**, shows it in a table, and lets users **edit, update and delete** records. Publish the whole project to a **GitHub repository**.

1. Learning Outcomes

By the end of this lab, you will be able to:

- Create a Spring Boot project with Spring Initializer and select the required dependencies.
- Install MySQL, create a database, and connect a Spring Boot application to it.
- Keep database passwords out of GitHub using an ignored configuration file.
- Use Lombok annotations (@Getter, @Setter, @NoArgsConstructor) instead of writing getters and setters by hand.
- Build the layers of a Spring MVC application: Entity, Repository, Service, Controller and View.
- Use postman to implement Edit, Update and Delete operations (CRUD).
- Verify every change directly in MySQL and push the project to GitHub using a feature branch and a Pull Request.
- In this lab sheet you need to complete full code by yourself.

2. Scenario

A company needs a small **Employee Management System**. An HR officer opens a web page, enters the details of an employee (name, email, phone, department, salary) and clicks Save. The data is stored in a **MySQL database** and appears immediately on a table below the form. Each row has an Edit button (to change the details) and a Delete button (to remove the employee). Every action must be reflected in the MySQL database.

Unlike an in-memory database, MySQL is a real database server: your data stays in the database even after the application is stopped and restarted. You will prove this during testing.

3. Prerequisites

- **JDK 17 or later** (JDK 21 recommended). Check with java -version.
- An IDE: IntelliJ IDEA, VS Code (with 'Extension Pack for Java' and 'Spring Boot Extension Pack') or Spring Tool Suite. Lombok must be supported by your IDE (see Task 1, step 6).
- MySQL Community Server 8.x (with MySQL Workbench) or XAMPP/MariaDB. You must know your MySQL root password. Task 2 explains the installation.
- Git installed and configured, and a GitHub account (Lab Sheets 02 and 03).

- An internet connection (Maven downloads the dependencies the first time).

4. Application Overview

The application follows the standard layered architecture. A browser request flows from top to bottom, and the response flows back up.

Layer	File	Responsibility
View	employees.html, edit-employee.html	Thymeleaf pages: the form and the table
Controller	EmployeeController.java	Receives HTTP requests, validates input, chooses the page to show
Service	EmployeeService.java	Business logic between controller and database
Repository	EmployeeRepository.java	Spring Data JPA interface that talks to MySQL (no SQL needed)
Entity or DTO	Employee.java or DTO class	Java class mapped to the employee's table; getters/setters by Lombok

URLs handled by the controller:

Method	URL	Purpose
GET	/employees	Show the form and the table of all employees (Read)
POST	/employees	Save a new employee (Create)
GET	/employees/{id}/edit	Show the edit form filled with the employee's data
POST	/employees/{id}/update	Save the changes (Update)
POST	/employees/{id}/delete	Remove the employee (Delete)

What does Lombok do? Lombok writes repetitive code for you at compile time. Compare:

```
// WITHOUT Lombok (you write all of this)
public class Employee {
    private String fullName;
    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }
    public Employee() { }
}

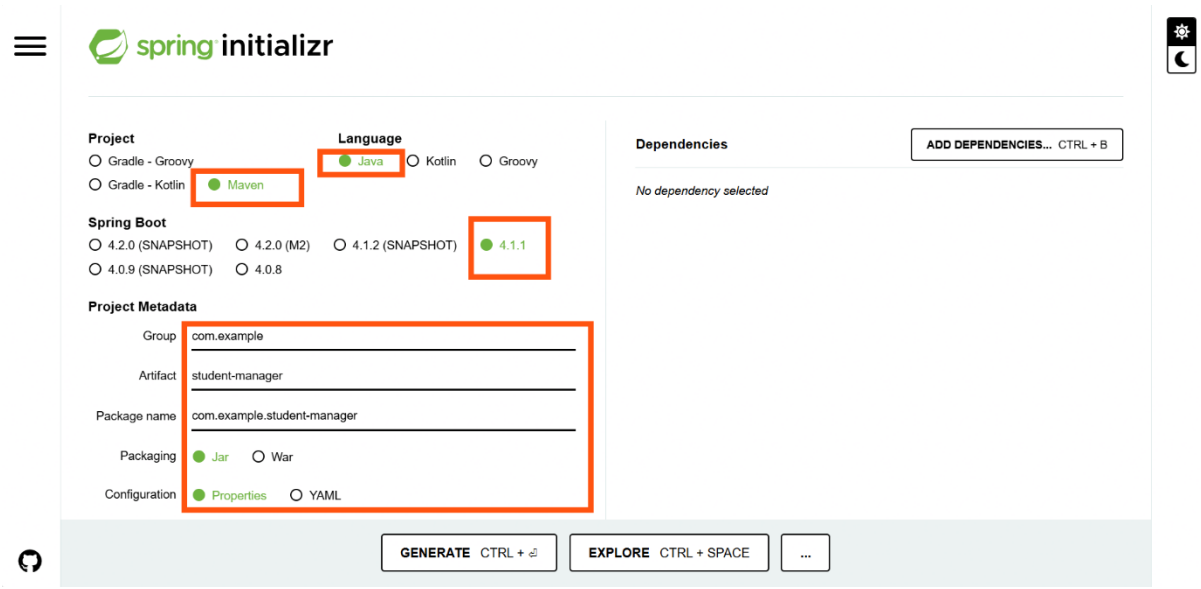
// WITH Lombok (same result, 3 annotations)
@Getter @Setter @NoArgsConstructor
public class Employee {
    private String fullName;
}
```

5. Tasks

Task 1: Create the Project with Spring Initializer

1. Open **https://start.spring.io** in your browser.
2. Fill in the **Project Metadata** exactly as shown:

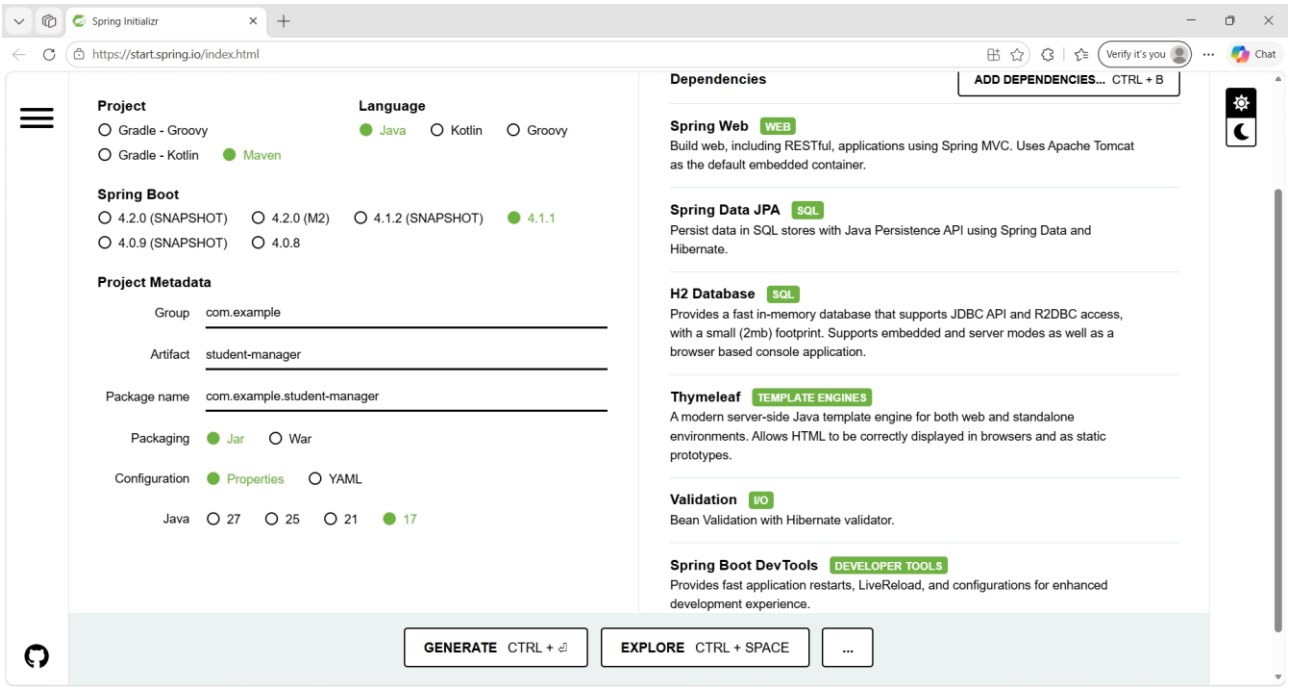
Setting	Value
Project	Maven
Language	Java
Spring Boot	The latest stable release (do not choose SNAPSHOT or M1/RC versions)
Group	com.example
Artifact / Name	employee-manager
Description	Employee Management System
Package name	com.example.employeemanager
Packaging	Jar
Java	21 (or 17 if 21 is not installed on your machine)



3. Click **ADD DEPENDENCIES** (or press Ctrl + B), search for each dependency below and add it. You must end up with **seven** dependencies:

Dependency (search name)	Added to pom.xml	Why we need it
Spring Web	spring-boot-starter-web (-webmvc in Boot 4)	Spring MVC, embedded Tomcat server, controllers
Spring Data JPA	spring-boot-starter-data-jpa	Hibernate + repositories: save/find/delete without SQL
MySQL Driver	mysql-connector-j	JDBC driver that lets Java connect to MySQL

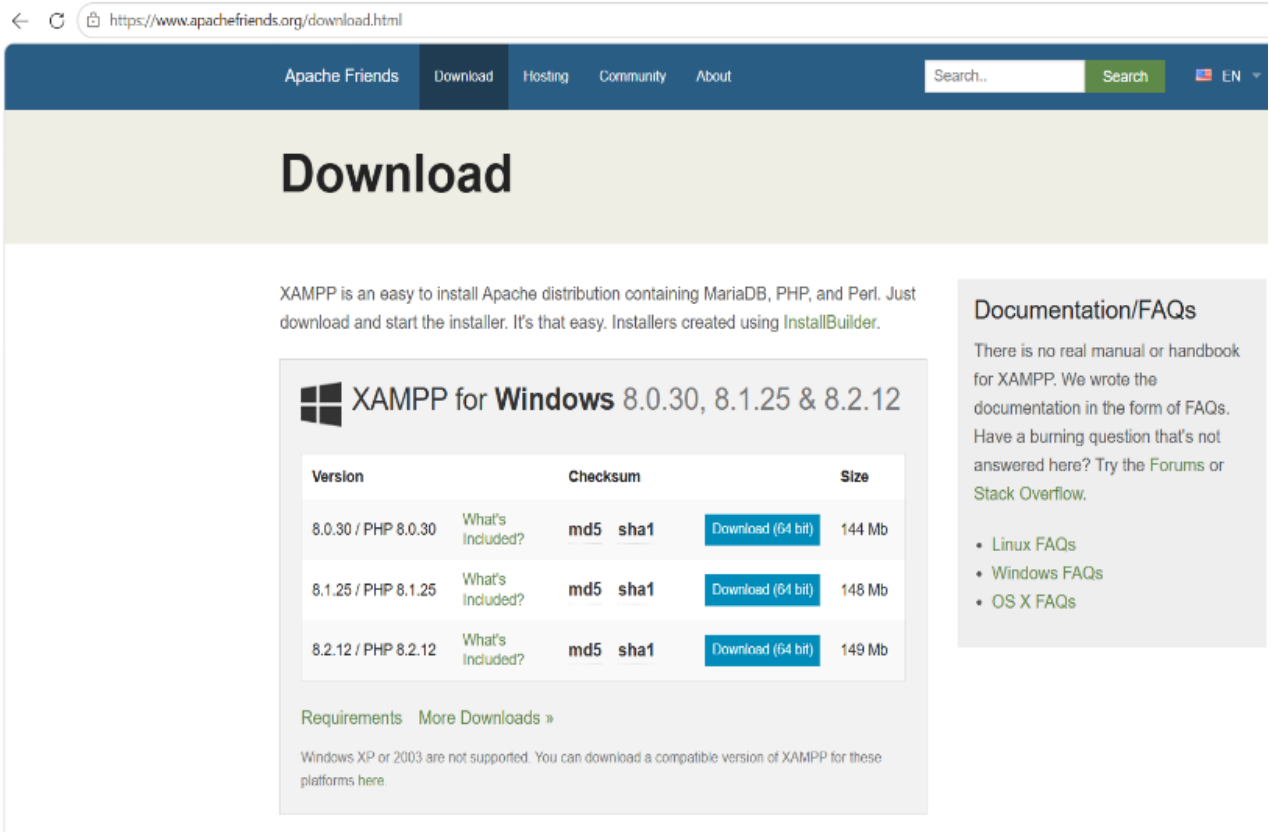
Dependency (search name)	Added to pom.xml	Why we need it
Validation	spring-boot-starter-validation	Annotations such as @NotBlank, @Email, @Positive
Lombok	lombok	Generates getters, setters and constructors automatically
Spring Boot DevTools	spring-boot-devtools	Automatic restart when you change code



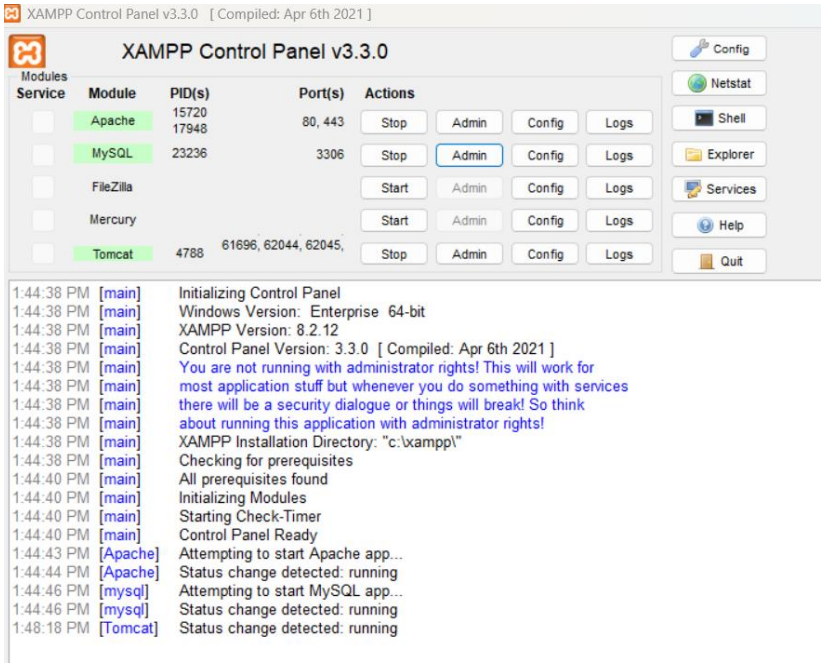
4. Click **GENERATE**. A file employee-manager.zip is downloaded. Extract it, then open the extracted employee-manager folder in your IDE (**File → Open Folder / Open Project**). Wait until the IDE finishes importing the Maven project.
5. Open pom.xml and confirm that all seven dependencies appear. A complete reference pom.xml is given in **Appendix A**.
6. **Enable Lombok in your IDE** so it does not show red errors on the methods generated:
   - **IntelliJ IDEA:** Settings → Build, Execution, Deployment → Compiler → Annotation Processors → tick **Enable annotation processing** (the Lombok plugin is built into recent versions).
   - **VS Code:** install/update the Extension Pack for Java, then run **Developer: Reload Window**.
   - **Eclipse / STS:** run the Lombok installer jar once, select your IDE, and restart.

Task 2: Install MySQL and Create the Database

1. **Install XAMPP Server** (skip if already installed): start the MySQL module in the XAMPP control panel; the root password is empty by default.)

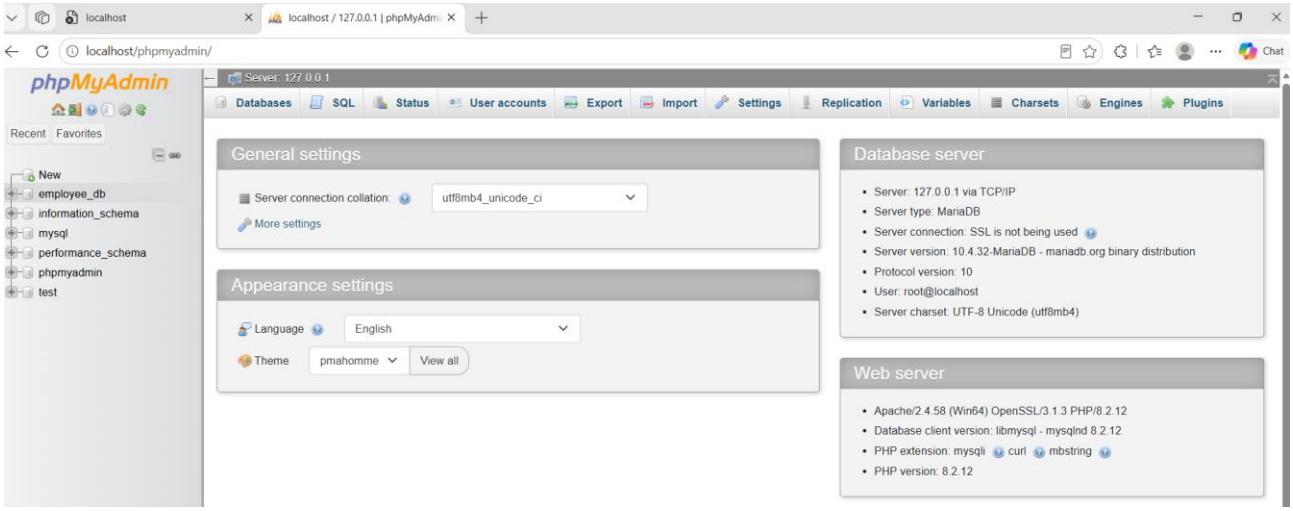


2. Make sure the MySQL service is **running** (Windows: Services → MySQL80 → Running; or check that the XAMPP MySQL module is green).



3. Create the database that the application will use and check that it exists:

```
CREATE DATABASE employee_db;
SHOW DATABASES;
```



Task 3: Create the GitHub Repository and Push the Initial Project

We push the project **before** adding any password, so secrets never enter the Git history.

1. On GitHub click **new repository**. Name it employee-manager-springboot, choose **Public** or **Private**, and **do not** tick 'Add a README' or '.gitignore'. Click **Create repository**.

2. In the project folder terminal run (replace <your-username>):

```
git init
git add .
git status                # target/ must NOT be listed
git commit -m "Initial Spring Boot project from Spring Initializer"
git branch -M main
git remote add origin https://github.com/<your-username>/employee-manager-springboot.git
git push -u origin main
```

3. Refresh the repository page and confirm that pom.xml, mvnw, src/ and .gitignore are visible.

Task 4: Configure the MySQL Connection

1. Open src/main/resources/application.properties and replace its content with:

```
spring.application.name=employee-manager

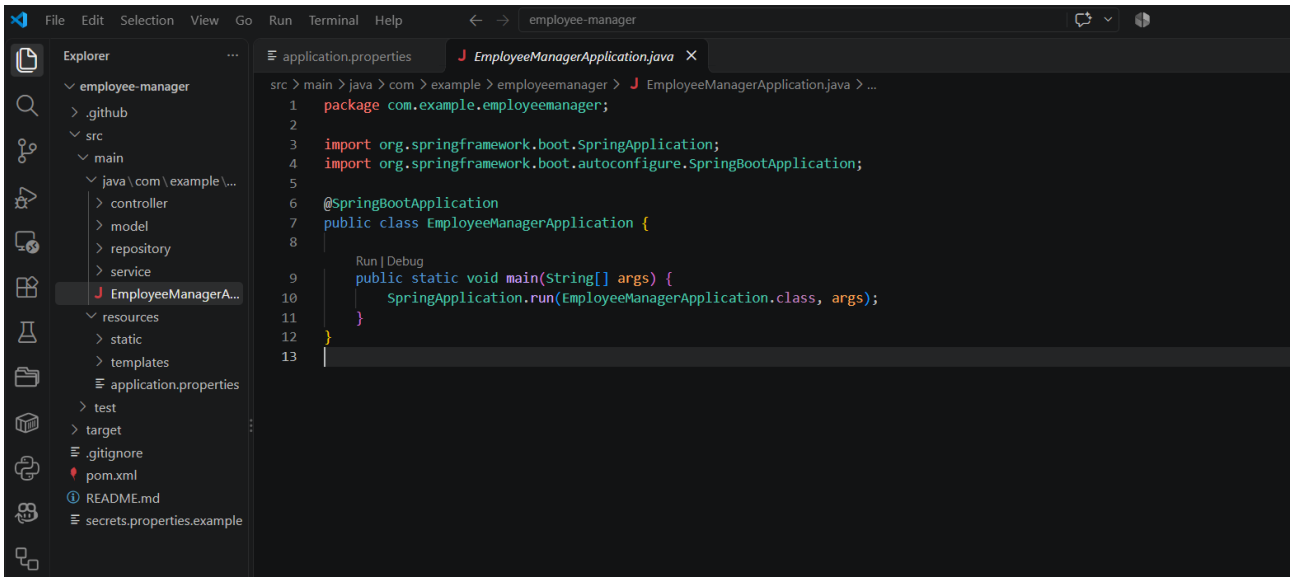
# ---- XAMPP MySQL connection ----
spring.datasource.url=jdbc:mysql://localhost:3306/employee_db?createDatabaseIfNotExist=true
&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=

# ---- JPA / Hibernate ----
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# ---- Thymeleaf ----
```

```
spring.thymeleaf.cache=false
```

2. Go to the File EmployeeManagerApplication.java and run it.



3. After that you will see **"HikariPool-1 - Start completed"** and **"Started EmployeeManagerApplication"** in the terminal. Open http://localhost:8080. A 'Whitelabel Error Page (404)' is expected because we have not created any page yet. Stop the server with **Ctrl + C**.

Property	Meaning
spring.datasource.url	jdbc:mysql://localhost:3306/employee_db = MySQL on this computer, port 3306, database employee_db. The extra options avoid common SSL / time-zone errors.
spring.config.import	Loads secrets.properties if it exists, so the username and password stay out of Git
spring.jpa.hibernate.ddl-auto=update	Hibernate creates/updates the employees table from your entity automatically
spring.jpa.show-sql=true	Prints the generated SQL in the terminal (useful for learning)
spring.thymeleaf.cache=false	Shows page changes immediately during development

Task 5: Create Entity with Lombok

1. Work on a feature branch from now on:

```
git switch -c feature/employee-crud
```

2. Create the packages model, repository, service and controller inside com.example.employeemanager. Then create the class Employee.java in the **model** package. Notice that there are **no getters or setters** in the file; Lombok generates them:

Annotation	Purpose
@Getter, @Setter	Lombok: creates getFullName(), setFullName(...), getSalary() ... for every field
@NoArgsConstructor	Lombok: creates the empty constructor that JPA and Thymeleaf need
@Entity, @Table	Maps the class to the employees table
@Id, @GeneratedValue	Primary key that MySQL generates automatically (AUTO_INCREMENT)
@Column(unique = true)	No two employees can have the same email
@NotBlank, @Email, @Pattern, @Positive	Validation rules checked when the form is submitted

> **Check:** In your IDE open the Outline / Structure view of Employee. You should see methods like getFullName() even though you did not write them. If not, revisit Task 1, step 6. Avoid @Data on JPA entities because its generated equals/hashCode can cause problems.

```
git add .
git commit -m "Add Employee entity using Lombok and validation rules"
```

Task 6: Create the Repository and Service

1. Create the interface EmployeeRepository in the **repository** package. Extending JpaRepository gives you save, findAll, findById, deleteById and more with **no implementation code**. The two extra methods are created by Spring from their names.

2. Create the class EmployeeService in the **service** package:

```
git add .
git commit -m "Add EmployeeRepository and EmployeeService"
```

Task 7: Create the Controller

Create EmployeeController in the **controller** package. Each method handles one CRUD operation (see the comments in the code):

> **Key ideas:** @Valid runs the validation annotations from the entity; BindingResult holds any errors; redirect: sends the browser to a fresh page after saving so that pressing Refresh does not submit the form twice (Post-Redirect-Get).

```
git add .
git commit -m "Add EmployeeController with create, update and delete"
```

Task 8: Run and Test the Application

1. Make sure MySQL is running, then start the application:
2. Open http://localhost:8080. You should see the registration form and an empty table.
3. Test every operation:

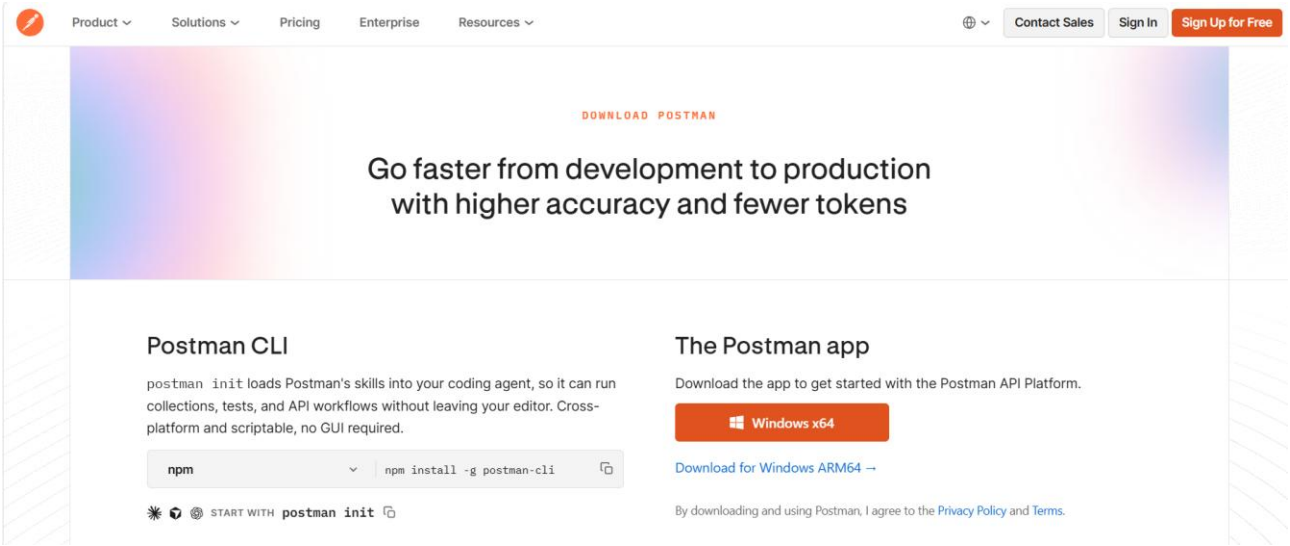
Operation	What to do	Expected result
Validation	Click **Save Employee** with all fields empty. Then enter an invalid email, a 5-digit phone number and a salary of 0.	Red error messages appear under the fields; nothing is saved
Create	Add at least **four** employees with different data.	Green message; employees appear in the table
Duplicate	Add an employee with an email that already exists.	Error 'This email is already registered'
Update	Click **Edit** on an employee, change the department and salary, click **Update Employee**.	Table shows the new values
Delete	Click **Delete** on one employee and confirm.	The row disappears; counter decreases

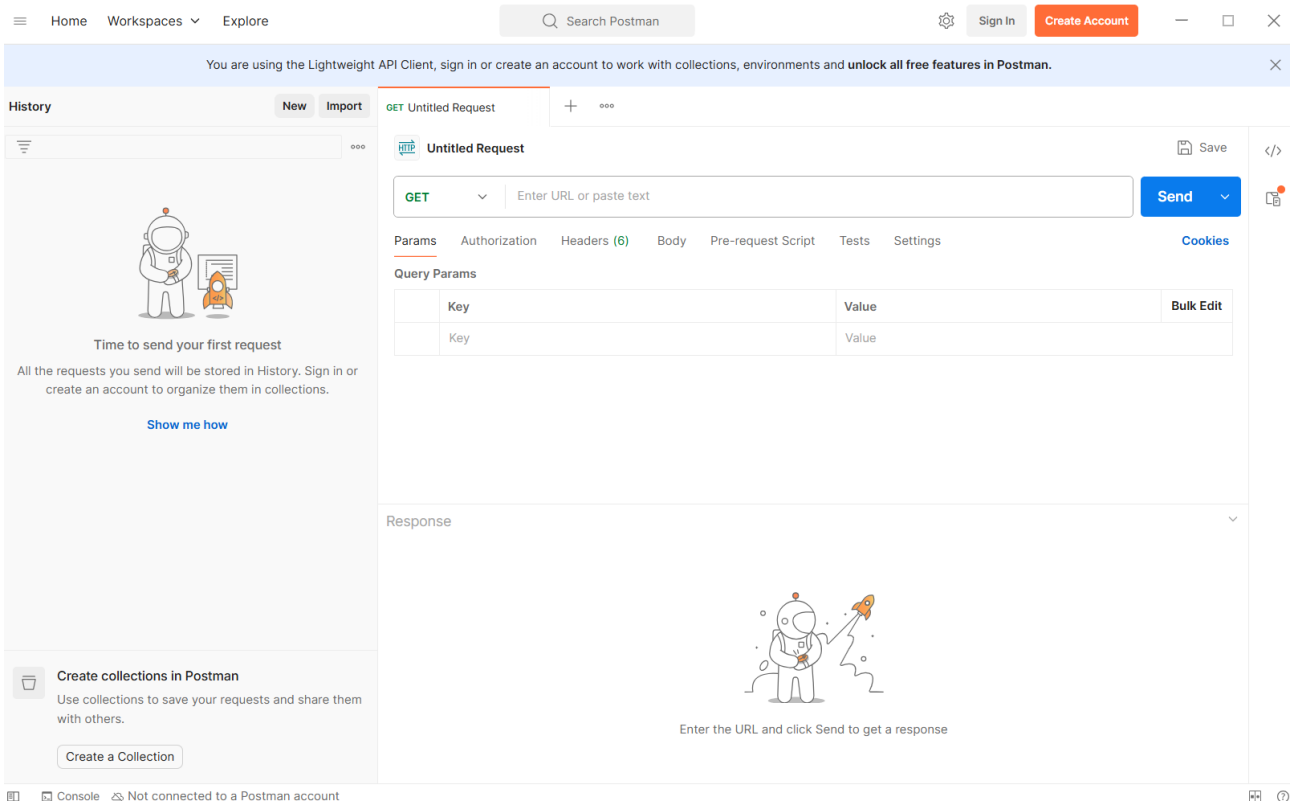
4. Verify **the data in MySQL.** Open a MySQL terminal (or MySQL Workbench) and run the statements below. Compare the result with the table on your web page. Then edit and delete a record on the web page and run the SELECT again to see the database change:

```
USE employee_db;
SHOW TABLES;
DESCRIBE employees;
SELECT * FROM employees;
```

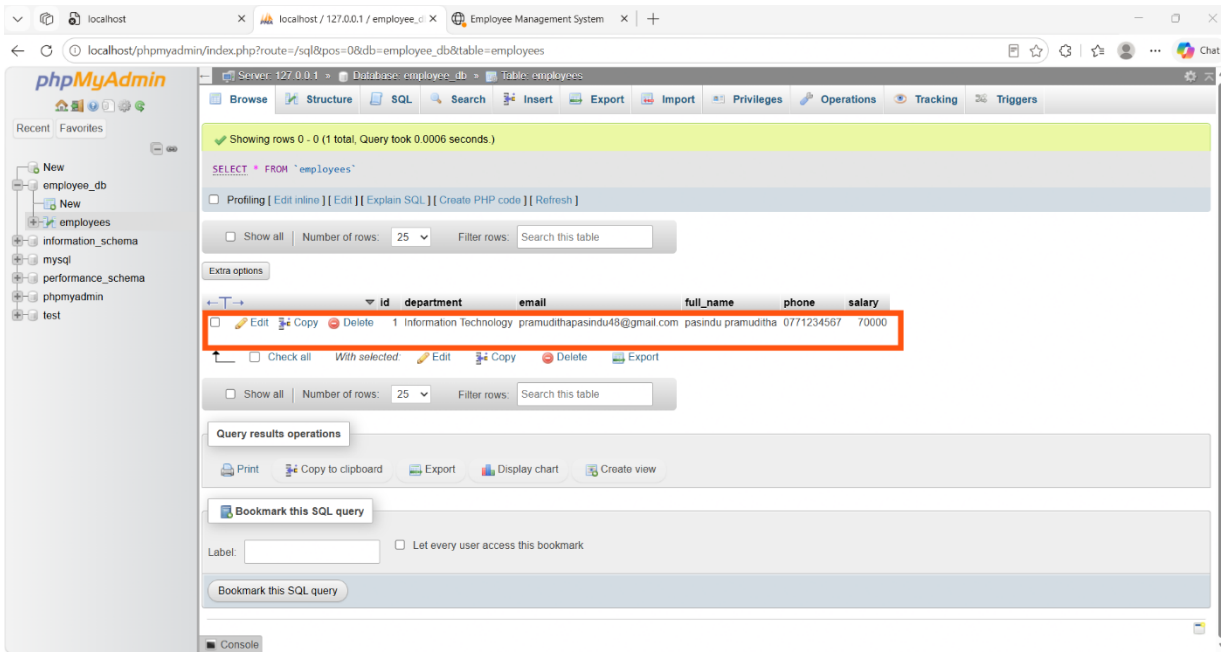
Task 9: Use postman to implement CRUD Operations

1. Install postman (if it did not install): [Download Postman | Get Started for Free]





2. Use postman to execute the CRUD Operations.
3. Check phpMyAdmin interface that shows the inserted, updated data in the table. Make sure to check all the CRUD Operations.



Task 10: Push to GitHub, Pull Request and Merge

1. Check the changes and push your feature branch:

```
git status              # secrets.properties must NOT be listed
git log --oneline
git push -u origin feature/employee-crud
```

2. On GitHub click **Compare & pull request**. Set **base: main** and **compare: feature/employee-crud**. Use the title ‘Add employee CRUD with MySQL and Lombok’, describe what you built, and click **Create pull request**.
3. Open **Files changed** and confirm that your Java files, templates, CSS and application.properties are included, and that **neither `secrets.properties` nor `target/`** appears. If secrets.properties appears, change your password immediately and ask your lecturer how to remove it from the history.
4. Click **Merge pull request → Confirm merge**.
5. Update your local main and confirm all files are present:

```
git switch main
git pull origin main
git log --oneline --graph
```

6. Add secrets.properties.example and a short README.md (what the project is and how to run it) if they are not in the repository yet, using a new branch and Pull Request.

Task 11 (Challenge): Search for Employees by Name

Add a search box above the table. Add this method to EmployeeRepository, call it from the service, and accept an optional keyword request parameter in the list method of the controller. Show only matching rows.

```
List<Employee> findByFullNameContainingIgnoreCase(String keyword);
```

6. Troubleshooting

Problem / error message	Solution
Communications link failure / Connection refused	MySQL is not running or the port is wrong. Start the MySQL service (or XAMPP MySQL) and check that the URL uses port 3306.
Access denied for user ‘root’@‘localhost’ (using password: YES)	The password in secrets.properties is wrong. Test it with mysql -u root -p.
Access denied for user ‘’@‘localhost’ (using password: **NO**)	secrets.properties was not found. It must be in the **project root** (next to pom.xml) and you must run the app from that folder.
Unknown database ‘employee_db’	Create it: CREATE DATABASE employee_db; (Task 2).
Public Key Retrieval is not allowed	Keep allowPublicKeyRetrieval=true in the JDBC URL (already included).
Red errors on getFullName() / ‘cannot find symbol’	Lombok is not active. Enable annotation processing / reload the IDE (Task 1, step 6) and reload the Maven project.
Port 8080 was already in use	Stop the other program, or add server.port=8081 to application.properties.

Problem / error message	Solution
Whitelabel error / ‘Error resolving template’	Template names must match exactly: employees.html, edit-employee.html, fragments.html in src/main/resources/templates.
./mvnw test fails	The test starts the full application, so MySQL must be running and secrets.properties must exist.
‘release version 21 not supported’	Your JDK is older than java.version in pom.xml. Install JDK 21 or change the value to 17.
target/ or secrets.properties appears in git status	Check .gitignore, then run git rm -r --cached target (or git rm --cached secrets.properties).

7. Reference

Useful MySQL statements

Statement	Purpose
SHOW DATABASES;	List all databases
USE employee_db;	Select the database
SHOW TABLES;	List tables in the selected database
DESCRIBE employees;	Show the columns of the table
SELECT * FROM employees;	Show all rows
DELETE FROM employees WHERE id = 3;	Delete one row manually

Useful commands

Command	Purpose
git switch -c <branch>	Create a branch
git add . / git commit -m "msg"	Stage and commit changes
git push -u origin <branch>	Upload a new branch
git pull origin main	Download the latest main

Annotations used in this lab

Annotation	Purpose
@SpringBootApplication	Marks the main class; enables auto-configuration
@Getter / @Setter / @NoArgsConstructor	Lombok: generate getters, setters and the empty constructor
@Entity / @Id	Maps a class / field to a database table / primary key
@Service	Marks a business-logic class
@Controller	Marks a class that returns pages (views)

Annotation	Purpose
@GetMapping / @PostMapping	Maps a URL and HTTP method to a method
@PathVariable	Reads a value from the URL (e.g. {id})
@ModelAttribute	Binds form data to a Java object
@Valid	Triggers validation of the object

Submission Requirements

- Submit as a PDF including all the screenshots and code.
- Include your name, student ID, date, course code, course name and your GitHub repository URL.
- The filename should be named as 'Labsheet05_{your student ID}.pdf'
- Deadline: 26th of October 2026 at 11.59 pm.